Owasp top 10

08 September 2026 01:54 AM

web application

OWASP	Vulnerability	Practical testing / attack types		Recommendation / Protection
	Broken Access Control	**1. Parameter tampering:** URL/request me id, user_id, account_id jaise identifiers change karke doosre user's object ko access karne ki koshish. **2. Guess/forced browsing:** /admin, /admin/users, /manage jaise restricted functions ko normal user session se request karna. **3. UI bypass:** browser ke buttons/menus ko ignore karke Burp se backend request directly bhejna. **4. HTTP method tampering:** GET se blocked operation ko POST/PUT/DELETE ke through try karna. **5. Horizontal privilege test:** User A → User B resource. **6. Vertical privilege test:** normal user → admin function.	**1. Broken Access Control** 1. Enforce authorization on the server side. 2. Follow a deny-by-default approach. 3. Use a centralized access-control policy. 4. Apply authorization checks to every sensitive object and function. 5. Enable security-event logging and alerting. 6. Implement rate limiting and abuse controls. 7. Invalidate the session when the user logs out. 8. Do not rely on client-side restrictions.	**1. Authorization server-side enforce karo.** **2. Deny-by-default follow karo.** **3. Centralized access-control policy use karo.** **4. Har sensitive object/function par authorization check lagao.** **5. Security-event logging + alerting enable karo.** **6. Rate limiting/abuse controls lagao.** **7. Logout par session invalidate karo.** **8. Client-side restrictions par trust mat karo.**
	Security Misconfiguration	**1. Verbose error:** invalid request dekar stack trace/version/path disclosure check. **2. Debug mode:** debug pages/config exposed hain ya nahi. **3. Default credentials:** test environment me default/admin credentials remain to nahi. **4. HTTP methods:** unnecessary OPTIONS/PUT/DELETE etc. enabled. **5. CORS:** overly broad origins/credentials . **6. Security headers:** CSP, HSTS, X-Content-Type-Options etc. review. **7. Exposed admin/management endpoint.** **8. Directory listing / backup files.**	**2. Security Misconfiguration** 1. Maintain a secure baseline/configuration standard. 2. Disable debug mode. 3. Remove or change default accounts and credentials. 4. Disable unnecessary services and features. 5. Restrict management interfaces. 6. Explicitly allowlist CORS origins. 7. Use safe error messages. 8. Implement secure headers and TLS configuration. 9. Perform regular configuration reviews.	**1. Secure baseline/configuration standard maintain karo.** **2. Debug disable karo.** **3. Default accounts/credentials remove/change karo.** **4. Unnecessary services/features disable karo.** **5. Management interface restrict karo.** **6. CORS explicitly allowlist karo.** **7. Safe error messages use karo.** **8. Secure headers and TLS configuration implement karo.** **9. Regular configuration review karo.**
	Software Supply Chain Failures	**1. Dependency inventory:** libraries/frameworks identify karo. **2. Version analysis:** old/vulnerable versions identify karo. **3. Transitive dependency review:** dependency ki dependency bhi check karo. **4. SBOM verification.** **5. Package source review:** trusted repository use ho rahi hai ya nahi. **6. CI/CD review:** build pipeline me unauthorized modification possible hai ya nahi. **7. Artifact integrity:** build/release artifacts signed/verified hain ya nahi.**	**3. Software Supply Chain Failures** 1. Maintain an SBOM. 2. Implement SCA scanning. 3. Pin and verify dependencies. 4. Use trusted repositories. 5. Protect the CI/CD pipeline. 6. Sign and verify build/release artifacts. 7. Define a dependency patching SLA. 8. Remove or upgrade unsupported components.	**1. SBOM maintain karo.** **2. SCA scanning implement karo.** **3. Dependencies pin/verify karo.** **4. Trusted repositories use karo.** **5. CI/CD pipeline protect karo.** **6. Build/release artifacts sign & verify karo.** **7. Dependency patch SLA define karo.** **8. Unsupported components remove/upgrade karo.**
	Cryptographic Failures	**1. Sensitive data over HTTP:** credentials/session/PII insecure transport par ja raha hai ya nahi. **2. TLS review:** weak protocol/cipher/certificate configuration. **3. Cookie security:** Secure/HttpOnly/SameSite flags. **4. Password storage:** plaintext/weak hashing indication. **5. Hardcoded secrets/keys:** application code/config me secrets. **6. Weak/deprecated algorithm usage. **7. Sensitive data unnecessarily stored.**	**4. Cryptographic Failures** 1. Use strong TLS for sensitive traffic. 2. Store passwords using dedicated password-hashing algorithms. 3. Keep cryptographic keys separate from application source code. 4. Use centralized key management. 5. Remove deprecated cryptographic mechanisms. 6. Minimize sensitive data. 7. Configure secure cookie attributes. 8. Maintain a key rotation and revocation process.	**1. Sensitive traffic ke liye strong TLS use karo.** **2. Passwords ko dedicated password-hashing algorithms se store karo. 3. Keys ko application source se separate rakho.** **4. Centralized key management use karo.** **5. Deprecated crypto remove karo.** **6. Sensitive data minimize karo.** **7. Secure cookie attributes configure karo.** **8. Key rotation/revocation process rakho.**
	Injection	**1. SQL injection:** parameters/search/filter inputs ko controlled test values se test karna. **2. NoSQL injection:** JSON/query operators ke handling test. **3. OS command injection:** user input command execution context tak pahunchta hai ya nahi. **4. LDAP injection.** **5. Template injection.** **6. Header-based injection.** **7. XML/XPath injection where applicable.** Testing me baseline request vs modified request compare karo aur backend error/behavior observe karo.	**5. Injection** 1. Use parameterized queries/prepared statements. 2. Do not concatenate user input into command strings. 3. Use allowlist validation where practical. 4. Apply context-aware output encoding. 5. Use safe framework APIs. 6. Apply least privilege to backend service accounts. 7. Avoid unnecessary interpreter or shell execution	**1. Parameterized queries/prepared statements use karo.** **2. User input ko command string me concatenate mat karo.** **3. Allowlist validation use karo jahan practical ho.** **4. Context-aware output encoding karo. 5 . Safe framework APIs use karo.** **6. Backend service accounts ko least privilege do.** **7. Unnecessary interpreters/shell execution avoid karo.**
	Insecure Design	**1. Business-logic abuse:** workflow ko expected sequence se bahar execute karna. **2. Trust-boundary testing:** client-side value par excessive trust. **3. Missing anti-automation controls.** **4. Missing transaction limits.** **5. Password-reset/account-recovery design weaknesses.** **6. Multi-step process me authorization missing.** **7. Threat-model gaps identify karna.**	**6. Insecure Design** 1. Perform threat modeling before development. 2. Define abuse cases. 3. Include security requirements in the architecture. 4. Enforce business rules on the server side. 5. Apply transaction limits and risk controls to sensitive workflows. 6. Make security design reviews mandatory.	**1. Threat modeling development se pehle karo.** **2. Abuse cases define karo.** **3. Security requirements architecture me include karo.** **4. Server-side business rules enforce karo.** **5. Sensitive workflows par transaction limits/risk controls lagao.** **6. Security design review mandatory karo.**
	Authentication Failures	**1. Login bypass attempts.** **2. Weak password policy.** **3. Brute-force/rate-limit testing in authorized lab.** **4. Session fixation.** **5. Session token reuse after logout.** **6. Password-reset workflow testing.** **7. MFA enforcement/flow review.** **8. Remember-me/session persistence review.** **9. Reauthentication for sensitive actions.**	**7. Authentication Failures** 1. Use a strong authentication mechanism. 2. Implement MFA where appropriate. 3. Rate-limit login attempts. 4. Rotate the session after authentication or privilege changes. 5. Invalidate the session/token on logout. 6. Implement secure password recovery. 7. Require reauthentication for sensitive actions. 8. Log and monitor authentication events.	**1. Strong authentication mechanism use karo.** **2. MFA implement karo where appropriate.** **3. Rate-limit login attempts.** **4. Session rotate karo after authentication/privilege change.** **5. Logout par session/token invalidate karo.** **6. Secure password recovery implement karo.** **7. Sensitive actions ke liye reauthentication use karo.** **8. Authentication events log/monitor karo.**
	Software or Data Integrity Failures	**1. Insecure update mechanism.** **2. Unsigned/unverified software artifacts . 3. CI/CD artifact manipulation possibility.** **4. Unsafe deserialization paths.** **5. Client/server data integrity assumptions . 6. Third-party package/artifact trust review.**	**8. Software or Data Integrity Failures** 1. Sign software and artifacts. 2. Verify signatures and integrity. 3. Strongly protect CI/CD access. 4. Avoid unsafe deserialization. 5. Maintain build provenance. 6. Make integrity checks mandatory in the deployment pipeline. 7. Apply least privilege.	**1. Software/artifacts sign karo.** **2. Signature/integrity verify karo.** **3. CI/CD access strongly protect karo.** **4. Unsafe deserialization avoid karo.** **5. Build provenance maintain karo.** **6. Deployment pipeline me integrity checks mandatory karo.** **7. Least privilege apply karo.**
	Security Logging and Alerting Failures	**1. Failed login generate karo → log banta hai ya nahi.** **2. Unauthorized admin access attempt → alert generate hota hai ya nahi.** **3. Privilege change → audit event verify.** **4. Password reset/account change → audit trail check.** **5. Input/security failures → logging verify.** **6. Log fields: timestamp, user, source, action, result, correlation ID.** **7. Check whether logs can be modified/deleted by normal users.**	**9. Security Logging and Alerting Failures** 1. Log security events centrally. 2. Integrate with a SIEM. 3. Create alerts for high-risk events. 4. Keep logs tamper-resistant. 5. Maintain time synchronization. 6. Define a log-retention policy. 7. Link monitoring with incident-response playbooks. 8. Do not unnecessarily store sensitive data in logs.	**1. Security events centrally log karo.** **2. SIEM integration karo.** **3. High-risk events par alerts banao.** **4. Logs tamper-resistant rakho.** **5. Time synchronization maintain karo.** **6. Log retention policy define karo.** **7. Monitoring + incident-response playbooks link karo.** **8. Sensitive data ko logs me unnecessarily store mat karo.**
	Mishandling of Exceptional Conditions	**1. Invalid input/state tests. 2. Missing parameters. 3. Timeout/connection-failure behavior.	**10. Mishandling of Exceptional Conditions** 1. Follow a fail-closed design. 2. Use centralized exception handling.	**1. Fail-closed design follow karo.** **2. Centralized exception handling use karo.** **3. Sensitive information error response me**

4. Unexpected server response. 5. Concurrent request/state handling. 6. Transaction failure halfway through workflow. 7. Error handling me fail-open behavior check. **Goal:** exception ke time security control bypass hota hai ya fail-closed?	3. Do not expose sensitive information in error responses. 4. Ensure transaction rollback and integrity. 5. Handle timeouts properly. 6. Implement concurrency and state controls. 7. Capture exceptional events in security logs.	**expose mat karo.** **4. Transaction rollback/integrity ensure karo.** **5. Timeouts properly handle karo.** **6. Concurrency/state controls implement karo.** **7. Exceptional events ko security logs me capture karo.**

API VAPT

OWASP	Vulnerability	Practical testing / attack types		Recommendation / Protection
API1:2023	Broken Object Level Authorization (BOLA)	**1. Parameter tampering:** id, user_id, account_id, order_id change karke doosre user ka object access karna. **2. Horizontal access:** User A ke token se User B ka object read/update/delete karna. **3. Object ID enumeration:** sequential/random object IDs ko test karna. **4. Nested object testing:** /users/101/orders/5001 me parent/child IDs manipulate karna. **5. HTTP method testing:** same object par GET/PUT/PATCH/DELETE authorization compare karna. **6. UUID/reference substitution:** valid reference ko another valid reference se replace karna.	**API1:2023 — Broken Object Level Authorization (BOLA)** 1. Perform server-side authorization checks for every object access. 2. Follow a deny-by-default approach. 3. Perform ownership and tenant validation on the backend. 4. Do not treat a client-supplied ID as proof of authorization. 5. Use a centralized authorization policy. 6. Strictly block cross-tenant access. 7. Log and alert on unauthorized access attempts.	**1. Har object access par server-side authorization check karo.** **2. Deny-by-default follow karo.** **3. Ownership/tenant validation backend par karo.** **4. Client-supplied ID ko authorization ka proof mat samjho.** **5. Centralized authorization policy use karo.** **6. Cross-tenant access ko strictly block karo.** **7. Unauthorized access attempts log + alert karo.**
API2:2023	Broken Authentication	**1. Missing authentication:** protected endpoint bina token call karna. **2. Invalid token:** modified/expired/revoked token test karna. **3. Token validation:** issuer, audience, expiry, signature validation verify karna. **4. Authentication bypass:** alternative endpoint/version/method se protected function access karna. **5. Brute-force protection:** authorized lab me repeated login attempts ke baad throttling/lockout check karna. **6. MFA bypass:** MFA ke baad direct sensitive endpoint access test. **7. Session/token lifecycle:** logout/revocation ke baad token reuse check.	**API2:2023 — Broken Authentication** 1. Use a strong authentication mechanism. 2. Validate token signature, issuer, audience, and expiration. 3. Use short-lived access tokens. 4. Securely manage the refresh-token lifecycle. 5. Rate-limit login attempts. 6. Implement MFA where appropriate. 7. Invalidate the session/token upon logout or revocation. 8. Centrally log authentication events.	**1. Strong authentication mechanism use karo.** **2. Token signature, issuer, audience and expiry validate karo.** **3. Short-lived access tokens use karo** **.4. Refresh-token lifecycle securely manage karo.** **5. Login attempts rate-limit karo.** **6. MFA where appropriate implement karo.** **7. Logout/revocation par session/token invalidate karo.** **8. Authentication events centrally log karo.**
API3:2023	Broken Object Property Level Authorization (BOPLA)	**1. Excessive data exposure:** API response me unnecessary fields mil rahe hain ya nahi. **2. Hidden/sensitive fields:** isAdmin, role, salary, internal_id, reset_token jaise fields response me check karna. **3. Mass assignment:** request body me normally allowed fields ke saath additional properties insert karke server behavior dekhna. **4. Read vs write authorization:** field read allowed hai but update allowed nahi hona chahiye—ye verify karna. **5. Role-based field testing:** different roles ke response compare karna.	**API3:2023 — Broken Object Property Level Authorization (BOPLA)** 1. Return only the fields that are required. 2. Use an explicit field allowlist/DTO. 3. Apply field-level authorization to sensitive fields. 4. Prevent mass assignment. 5. Do not allow server-controlled properties to be writable by the client. 6. Appropriately restrict response schemas for different roles.	**1. Response me only required fields return karo.** **2. Explicit field allowlist/DTO use karo.** **3. Sensitive fields ke liye field-level authorization lagao.** **4. Mass assignment prevent karo.** **5. Server-controlled properties ko client se writable mat rakho.** **6. Different roles ke liye response schema appropriately restrict karo.**
API4:2023	Unrestricted Resource Consumption	**1. Rate-limit testing:** repeated requests controlled lab me bhejkar throttling check karna. **2. Large payload:** excessive JSON/body size submit karna. **3. Pagination abuse:** very large limit/page_size values test karna **.4. Expensive operation:** report/search/export jaise resource-heavy APIs test karna. **5. Concurrent request testing:** controlled parallel requests se resource handling observe karna. **6. File upload:** size/type/count limits verify karna. **7. Timeout testing:** slow/long-running requests par limits check karna.	**API4:2023 — Unrestricted Resource Consumption** 1. Implement per-user, per-IP, and per-token rate limiting. 2. Define request and body size limits. 3. Set a maximum pagination limit. 4. Enforce file upload size and count limits. 5. Implement timeouts and concurrency limits. 6. Define resource quotas. 7. Protect expensive APIs with authorization and throttling. 8. Enable monitoring and alerting for abnormal resource consumption.	**1. Per-user/IP/token rate limiting lagao.** **2. Request/body size limits define karo.** **3. Pagination maximum set karo.** **4. File upload size/count limits rakho.** **5. Timeouts and concurrency limits implement karo.** **6. Resource quotas define karo.** **7. Expensive APIs ko authorization + throttling se protect karo.** **8. Abnormal consumption par monitoring/alerting enable karo.**
API5:2023	Broken Function Level Authorization (BFLA)	**1. Admin endpoint guessing:** /admin, /admin/users, /management etc. ko low-privileged token se request karna. **2. HTTP method tampering:** GET/POST/PUT/PATCH/DELETE ke authorization differences check karna. **3. Endpoint substitution:** /user/... ko /admin/... pattern se test karna. **4. Role switching:** normal user vs manager vs admin requests compare karna. **5. Hidden API testing:** frontend me hidden function ko directly backend se call karna. **6. Version testing:** old API version me privileged function accessible hai ya nahi.	**API5:2023 — Broken Function Level Authorization (BFLA)** 1. Enforce function-level authorization on the server side. 2. Centralize RBAC/ABAC. 3. Perform role/permission checks for every privileged function. 4. Do not consider a hidden UI button a security control. 5. Retire old and deprecated endpoints. 6. Log and alert on unauthorized function calls.	**1. Function-level authorization server-side enforce karo.** **2. RBAC/ABAC centralized rakho.** **3. Har privileged function par role/permission check karo.** **4. Hidden UI button ko security control mat samjho.** **5. Old/deprecated endpoints retire karo.** **6. Unauthorized function calls log + alert karo.**
API6:2023	Unrestricted Access to Sensitive Business Flows	**1. Account creation automation:** repeated creation workflow test. **2. Login/OTP abuse:** automated repeated attempts ke protection controls check karna. **3. Password reset abuse:** excessive/reset automation test. **4. Coupon/promotion workflow:** repeated redemption logic test. **5. Booking/order workflow:** same business action repeatedly execute karna. **6. Transaction workflow:** limits, step-up verification and anti-bot controls verify karna. **7. Workflow bypass:** expected sequence ke steps skip karke next API directly call karna.	**API6:2023 — Unrestricted Access to Sensitive Business Flows** 1. Identify sensitive business flows. 2. Implement rate limiting and quotas. 3. Implement anti-automation and bot controls. 4. Require step-up verification for risky actions. 5. Define business transaction limits. 6. Validate workflow state on the server side. 7. Implement fraud and risk monitoring.	**1. Sensitive business flows identify karo.** **2. Rate limiting + quotas use karo.** **3. Anti-automation/bot controls implement karo.** **4. Step-up verification for risky actions.** **5. Business transaction limits define karo.** **6. Server-side workflow state validate karo** **7. Fraud/risk monitoring implement karo.**
API7:2023	Server-Side Request Forgery (SSRF)	**1. URL parameter testing:** API jo user-supplied URL fetch karti hai us input ko controlled lab endpoint se test karna. **2. Webhook testing:** callback/webhook destination validation check karna. **3. Import/fetch function:** remote resource import feature test karna. **4. Redirect handling:** server redirects ko follow karta hai ya nahi. **5. Protocol validation:** HTTP/HTTPS ke alawa unexpected schemes accept hote hain ya nahi. **6. Internal destination protection:** lab network me internal-only service ko application se reachable hai ya nahi.	**PI7:2023 — Server-Side Request Forgery (SSRF)** 1. Use a destination allowlist. 2. Do not blindly fetch user-supplied URLs. 3. Restrict outbound network egress. 4. Block internal/private destinations where applicable. 5. Strictly validate redirect handling. 6. Restrict allowed schemes and ports. 7. Provide network isolation for SSRF-prone services.	**1. Destination allowlist use karo.** **2. User-supplied URLs ko blindly fetch mat karo.** **3. Outbound network egress restrict karo.** **4. Internal/private destinations block karo where applicable.** **5. Redirect handling strictly validate karo.** **6. Allowed schemes/ports restrict karo.** **7. SSRF-prone service ko network isolation do.**

API8:2023	Security Misconfiguration	**1. Verbose errors:** invalid API request bhejkar stack trace/version/path disclosure check. **2. CORS review:** allowed origins/credentials verify. **3. Debug endpoints:** Swagger/debug/management interfaces exposed hain ya nahi **4. Unnecessary methods:** OPTIONS/PUT/DELETE etc. unnecessarily enabled hain ya nahi. **5. Default settings/credentials review.** **6. Security headers/TLS configuration review.** **7. Old API versions or test endpoints exposed.**	**API8:2023 — Security Misconfiguration** 1. Maintain a hardened API configuration baseline. 2. Disable debug mode. 3. Remove or change default credentials. 4. Explicitly allowlist CORS. 5. Disable unused endpoints and HTTP methods. 6. Restrict management interfaces. 7. Use safe error responses. 8. Implement API gateway and TLS hardening.	**1. Hardened API configuration baseline maintain karo.** **2. Debug mode disable karo.** **3. Default credentials remove/change karo.** **4. CORS explicitly allowlist karo.** **5. Unused endpoints/methods disable karo.** **6. Management interfaces restrict karo.7. Safe error responses use karo.8. API gateway + TLS hardening implement karo.**
API9:2023	Improper Inventory Management	**1. API discovery:** documented API vs actually deployed API compare karna. **2. Version enumeration:** /v1, /v2, /v3 etc. check karna. **3. Shadow APIs:** undocumented endpoints identify karna. **4. Deprecated API testing:** old versions still accessible hain ya nahi. **5. Swagger/OpenAPI exposure:** documentation publicly exposed hai ya nahi. **6. Host discovery:** dev/test/staging API accidentally internet-facing hai ya nahi. **7. Endpoint ownership review:** every API ka owner/data classification available hai ya nahi.	**API9:2023 — Improper Inventory Management** 1. Maintain a complete API inventory. 2. Use an API gateway/catalog. 3. Define an API version lifecycle. 4. Formally retire deprecated APIs. 5. Continuously discover shadow APIs. 6. Isolate development and test environments from production. 7. Document the API owner, data type, and authentication model.	**1. Complete API inventory maintain karo.** **2. API gateway/catalog use karo.** **3. Version lifecycle define karo.** **4. Deprecated APIs formally retire karo.** **5. Shadow APIs continuously discover karo.** **6. Dev/test environments production se isolate karo.** **7. API owner, data type and authentication model document karo.**
API10:2023	Unsafe Consumption of APIs	**1. Third-party API trust review:** external response ko automatically trusted maana ja raha hai ya nahi. **2. Response validation:** malformed/unexpected fields ke handling ko controlled test environment me verify karna. **3. Schema mismatch:** expected datatype ke against unexpected response test **.4. Redirect handling:** third-party service ke redirects safely handled hain ya nahi. **5. TLS/certificate validation review.** **6. Timeout/error handling:** external API unavailable ho to application safe behavior rakhti hai ya nahi. **7. External API data injection:** third-party returned data downstream interpreters me unsafe way se use ho rahi hai ya nahi.	**API10:2023 — Unsafe Consumption of APIs** 1. Treat third-party API responses as untrusted input. 2. Implement strict schema and data validation. 3. Enforce TLS and certificate verification. 4. Define timeouts and retry limits. 5. Apply least privilege to external integrations. 6. Handle third-party API failures safely. 7. Apply appropriate validation and encoding before passing data to downstream SQL, HTML, command, or template sinks. 8. Continuously monitor third-party dependencies and integrations.	**1. Third-party API responses ko untrusted input treat karo** **.2. Strict schema/data validation implement karo.** **3. TLS/certificate verification enforce karo.** **4. Timeouts and retry limits define karo.** **5. External integrations ko least privilege do.** **6. Third-party API failures safely handle karo.** **7. Downstream SQL/HTML/command/template sinks se pehle appropriate validation/encoding karo.** **8. Third-party dependencies and integrations continuously monitor karo.**

From <https://chatgpt.com/c/6a9ecae4-2c24-83ee-ace2-797d21d9eea6>

API VAPT mein ek important distinction

API testing mein **API1, API3 aur API5** ko alag rakhna bahut important hai:

Category	Main Question
API1 – BOLA	Kya main kisi doosre user's object ko access kar sakta hoon?
API3 – BOPLA	Kya main object ke unauthorized properties/fields read ya modify kar sakta hoon?
API5 – BFLA	Kya main aisa function/API operation chala sakta hoon jo meri role ke liye allowed nahi hai?

LLM VAPT

OWASP	Vulnerability	Practical testing / attack types		Recommendation / Protection
LLM01:2025	Prompt Injection	**1. Direct prompt injection:** user prompt ke through model ke intended behavior/instructions ko alter karne ki koshish. **2. Instruction override:** system/developer instructions ko ignore karne ke liye conflicting instructions dena. **3. Context manipulation:** prompt mein fake roles, priorities ya authority introduce karna. **4. Indirect prompt injection:** RAG/document/web content mein malicious instructions embed karke model se unhe follow karwane ki koshish. **5. Multi-turn injection:** multiple conversations ke through model behavior gradually manipulate karna. **6. Tool-oriented injection:** model ko unintended tool/function call karne ke liye influence karna.	**1. Prompt Injection** 1. Logically separate user input from trusted instructions. 2. Apply least privilege to tool access. 3. Do not use model output as an authorization mechanism. 4. Treat external and RAG content as untrusted. 5. Apply independent policy validation to tool calls. 6. Require human approval or confirmation for high-impact actions. 7. Implement prompt-injection detection and monitoring.	**1. User input aur trusted instructions ko logically separate rakho.** **2. Least-privilege tool access do.** **3. Model output ko authorization mechanism mat banao.** **4. External/RAG content ko untrusted treat karo.** **5. Tool calls par independent policy validation lagao.** **6. High-impact actions ke liye human approval/confirmation use karo.** **7. Prompt-injection detection + monitoring implement karo.**
LLM02:2025	Sensitive Information Disclosure	**1. Sensitive-data prompting:** model se confidential information reveal karne ki controlled lab testing. **2. Context leakage:** current conversation/context mein doosre user's confidential data exposure check. **3. RAG leakage:** restricted documents retrieve ho rahe hain ya nahi. **4. PII exposure:** names, email, financial, credentials, internal data output mein appear ho raha hai ya nahi. **5. Memory/session isolation:** ek session/user ka information doosre context mein accessible hai ya nahi. **6. Error/debug disclosure:** backend errors, API responses ya traces mein secrets leak ho rahe hain ya nahi.	**2. Sensitive Information Disclosure** 1. Apply data classification and minimization. 2. Do not unnecessarily send sensitive data into the model context. 3. Enforce access control for RAG. 4. Use PII and secret detection and redaction. 5. Enforce tenant and session isolation. 6. Do not hardcode secrets in prompts or system context. 7. Implement output DLP controls.	**1. Data classification + minimization karo.** **2. Sensitive data ko model context mein unnecessary mat bhejo.** **3. RAG access-control enforce karo.** **4. PII/secret detection and redaction use karo.** **5. Tenant/session isolation enforce karo.** **6. Secrets ko prompts/system context mein hardcode mat karo.** **7. Output DLP controls implement karo.**
LLM03:2025	Supply Chain	**1. Model provenance:** model/source verify karo. **2. Third-party model/plugin/SDK inventory.** **3. Dependency vulnerability review.** **4. Model/package integrity:** downloaded model/artifact tampered to nahi. **5. Dataset/source trust review.** **6. Build/deployment pipeline security.** **7. Dependency substitution/typosquatting risks review.**	**3. Supply Chain** 1. Use trusted model repositories. 2. Verify model and artifact integrity. 3. Maintain an SBOM and dependency inventory. 4. Evaluate third-party models and plugins. 5. Protect the CI/CD pipeline. 6. Pin and verify dependencies. 7. Maintain model provenance and version tracking.	**1. Trusted model repositories use karo.** **2. Model/artifact integrity verify karo.** **3. SBOM/dependency inventory maintain karo.** **4. Third-party models/plugins evaluate karo.** **5. CI/CD pipeline protect karo.** **6. Dependencies pin/verify karo.** **7. Model provenance aur version tracking maintain karo.**
LLM04:2025	Data and Model Poisoning	**1. Training-data integrity:** unauthorized/malicious records identify karo. **2. Fine-tuning dataset review:** malicious samples/instructions detect karo. **3. RAG data poisoning:** indexed documents mein manipulated content check karo. **4. Embedding poisoning:** malicious content retrieval ranking ko influence kar raha hai ya nahi. **5. Backdoor behavior:** specific trigger/input par abnormal model behavior test karo. **6. Data-source trust:** external ingestion pipeline verify karo.	**4. Data and Model Poisoning** 1. Maintain provenance for training and fine-tuning data. 2. Validate and sanitize datasets. 3. Use trusted data sources. 4. Implement data integrity, signing, and versioning. 5. Secure the RAG ingestion pipeline. 6. Review and approve dataset changes. 7. Maintain model-behavior baselines and regression testing.	**1. Training/fine-tuning data provenance maintain karo.** **2. Dataset validation and sanitization karo.** **3. Trusted sources use karo.** **4. Data integrity/signing/versioning implement karo.** **5. RAG ingestion pipeline secure karo.** **6. Dataset changes review/approve karo.** **7. Model behavior baseline + regression testing maintain karo.**
LLM05:2	Improper Output	**1. XSS-style testing:** model output directly HTML/UI mein render ho raha	**5. Improper Output Handling**	**1. LLM output ko untrusted input treat karo.**

025	Handling	hai ya nahi. **2. SQL/command sink:** LLM-generated output backend query/command mein directly pass ho raha hai ya nahi. **3. Code execution:** generated code automatically execute to nahi hota. **4. URL/output injection:** model-generated URLs/markup blindly trusted to nahi. **5. Structured output manipulation:** JSON/function arguments validate ho rahe hain ya nahi. **6. Markdown/HTML rendering:** unsafe content browser/client mein interpreted to nahi.	1. Treat LLM output as untrusted input. 2. Apply context-specific encoding and sanitization. 3. Use parameterization for SQL and command inputs. 4. Do not automatically execute generated code. 5. Validate structured output against a schema. 6. Independently authorize and policy-check tool arguments.	**2. Context-specific encoding/sanitization karo.3. SQL/command inputs ke liye parameterization use karo.** **4. Generated code ko automatically execute mat karo.** **5. Structured output schema validate karo.** **6. Tool arguments ko independent authorization/policy checks se validate karo.**
LLM06:2025	Excessive Agency	**1. Tool inventory:** model kin tools/functions ko call kar sakta hai. **2. Excessive permissions:** model ko required se zyada privileges diye gaye hain ya nahi. **3. Autonomous action:** model sensitive action human approval ke bina execute kar sakta hai ya nahi. **4. Chained actions:** ek low-risk request se multiple high-impact tool calls possible hain ya nahi. **5. Parameter manipulation:** generated tool arguments insufficiently checked hain ya nahi. **6. Cross-system access:** model unnecessary external systems/resources access kar sakta hai ya nahi.	**6. Excessive Agency** 1. Apply least privilege. 2. Maintain a tool/function allowlist. 3. Require human approval for high-risk actions. 4. Independently authorize every tool call. 5. Validate tool arguments against a schema. 6. Limit the blast radius. 7. Enable tool-execution logging and monitoring.	**1. Least privilege apply karo.** **2. Tool/function allowlist maintain karo.** **3. High-risk actions ke liye human approval rakho.** **4. Every tool call ko independently authorize karo.** **5. Tool arguments schema-validate karo.** **6. Blast radius limit karo.** **7. Tool execution logging + monitoring enable karo.**
LLM07:2025	System Prompt Leakage	**1. System-instruction disclosure testing:** model se hidden instructions disclose karne ke attempts. **2. Indirect extraction:** conversation/context manipulation ke through hidden instructions reveal hote hain ya nahi. **3. Role/format tricks:** output formatting ke through hidden configuration expose hoti hai ya nahi. **4. Error-based disclosure:** malformed requests se system/developer instructions leak hoti hain ya nahi. **5. Secret placement review:** API keys/passwords/confidential policy system prompt mein stored hain ya nahi.	**7. System Prompt Leakage** 1. Do not treat the system prompt as a secret-storage mechanism. 2. Do not store secrets in prompts. 3. Enforce sensitive authorization logic in backend services. 4. Ensure prompt disclosure does not create catastrophic impact. 5. Separate sensitive configuration from the model context.	**1. System prompt ko secret-storage mechanism mat samjho.** **2. Secrets prompt mein store mat karo.** **3. Sensitive authorization logic backend services mein enforce karo.** **4. Prompt disclosure ko catastrophic impact na banne do.** **5. Sensitive configuration ko model context se separate rakho.**
LLM08:2025	Vector and Embedding Weaknesses	**1. RAG authorization test:** User A restricted document ko retrieve kar sakta hai ya nahi. **2. Cross-tenant retrieval:** tenant A ke query results mein tenant B data appear hota hai ya nahi. **3. Document poisoning:** malicious/incorrect indexed content retrieval influence kar raha hai ya nahi. **4. Metadata manipulation:** document metadata/ACLs tamperable hain ya nahi. **5. Embedding collision/similarity issues:** unintended documents retrieval mein aa rahe hain ya nahi. **6. Retrieval boundary testing:** top-k/filters ke through restricted content expose to nahi.	**8. Vector and Embedding Weaknesses** 1. Enforce authorization at retrieval time. 2. Protect document-level ACL metadata. 3. Enforce tenant isolation. 4. Validate and sanitize ingested content. 5. Use a trusted embedding pipeline. 6. Protect metadata integrity. 7. Filter retrieval results according to user permissions.	**1. Retrieval-time authorization enforce karo.** **2. Document-level ACL metadata protect karo.** **3. Tenant isolation enforce karo.** **4. Ingested content validate/sanitize karo.** **5. Trusted embedding pipeline use karo.** **6. Metadata integrity protect karo.** **7. Retrieval results ko user permissions ke against filter karo.**
LLM09:2025	Misinformation	**1. Factuality testing:** known-answer questions ke against model response compare karo. **2. Hallucination testing:** model unsupported facts invent karta hai ya nahi. **3. RAG grounding:** answer retrieved source se actually supported hai ya nahi. **4. Citation verification:** model citations/source references valid hain ya nahi. **5. Confidence testing:** incorrect answer ko excessive certainty ke saath present karta hai ya nahi. **6. High-impact domain testing:** sensitive decision workflows mein unsupported output use ho raha hai ya nahi.	**9. Misinformation** 1. Use grounding/RAG where appropriate. 2. Implement source verification. 3. Require human review for high-impact decisions. 4. Communicate confidence and uncertainty appropriately. 5. Maintain automated factuality and regression tests. 6. Do not use model output as the sole authoritative source.	**1. Grounding/RAG use karo where appropriate.** **2. Source verification implement karo.** **3. High-impact decisions ke liye human review rakho.** **4. Confidence/uncertainty appropriately communicate karo.** **5. Automated factuality/regression tests maintain karo.** **6. Model output ko sole authoritative source mat banao.**
LLM10:2025	Unbounded Consumption	**1. Large-input testing:** excessive prompt/context size handling check. **2. Large-output testing:** maximum output/token limits verify karo. **3. Repeated request testing:** controlled lab mein rate limiting/throttling check karo. **4. Expensive operation testing:** repeated tool/model calls ka cost/resource impact check karo. **5. Recursive/agent loop:** workflow uncontrolled repeated calls kar sakta hai ya nahi. **6. File/context processing limits:** very large uploads/context windows handle kaise hote hain. **7. Quota bypass:** user/IP/API key limits properly enforce ho rahe hain ya nahi.	**10. Unbounded Consumption** 1. Set input/token limits. 2. Set output/token limits. 3. Implement per-user/API-key rate limiting. 4. Define model/tool call budgets. 5. Set maximum iterations for agent loops. 6. Set file/context size limits. 7. Implement cost/resource monitoring and alerting. 8. Use quotas and circuit breakers.	**1. Input/token limits set karo.** **2. Output/token limits set karo.** **3. Per-user/API-key rate limiting implement karo.** **4. Model/tool call budgets define karo.** **5. Agent loops ke maximum iterations rakho.** **6. File/context size limits rakho.** **7. Cost/resource monitoring + alerting implement karo.** **8. Quotas and circuit breakers use karo.**

From <https://chatgpt.com/c/6a9ecae4-2c24-83ee-ace2-797d21d9eea6>

Mobile VAPT

OWASP	Vulnerability	Practical testing / attack types		Recommendation / Protection
M1:2024	Improper Credential Usage	**1. Hardcoded credentials:** APK/IPA, resources, configs aur source/decompiled code mein API keys, passwords, tokens, certificates search karo. **2. Insecure credential storage:** Android SharedPreferences/files ya iOS UserDefaults/plain files mein credentials stored hain ya nahi. **3. Embedded secrets:** production app mein reusable service/API secrets present hain ya nahi. **4. Credential leakage through logs:** login/token-related values Android Logcat ya iOS logs mein aa rahe hain ya nahi. **5. Weak token handling:** long-lived access tokens improperly stored/reused ho rahe hain ya nahi. **6. Backup/extraction test:** app data backup/extraction ke baad sensitive credentials accessible hain ya nahi.	**1. Improper Credential Usage** 1. Do not hardcode long-lived secrets in the application. 2. Use Android Keystore / iOS Keychain. 3. Use short-lived access tokens. 4. Securely manage the refresh-token lifecycle. 5. Do not log credentials or tokens. 6. Maintain a mechanism to revoke and rotate compromised credentials. 7. Keep backend secrets on the server side	**1. Long-lived secrets app ke andar hardcode mat karo.** **2. Android Keystore / iOS Keychain use karo.** **3. Short-lived access tokens use karo.** **4. Refresh-token lifecycle securely manage karo.** **5. Logs mein credentials/tokens mat likho.** **6. Compromised credentials ko revoke/rotate karne ka mechanism rakho.** **7. Backend secrets ko server-side rakho.**
M2:2024	Insecure Authentication/Authorization	**1. Authentication bypass:** protected screen/API ko authentication ke bina invoke karne ki koshish. **2. Token manipulation:** invalid/expired session token ke behavior ko test karo. **3. Session reuse:** logout ke baad old session/token valid hai ya nahi **.4. Client-side authorization:** app sirf UI hide karke admin function protect to nahi kar rahi. **5. Role testing:** normal user aur privileged user ke requests compare karo. **6. Biometric flow:** biometric success/failure ke baad sensitive action ka server-side authorization check verify karo.	**2. Insecure Authentication/Authorization** 1. Enforce authentication and authorization on the server side. 2. Do not treat the client as a trusted boundary. 3. Maintain a strong session/token lifecycle. 4. Properly implement logout and revocation. 5. Use reauthentication/step-up authentication for sensitive actions.	**1. Authentication + authorization server-side enforce karo.** **2. Client ko trust boundary mat samjho.** **3. Strong session/token lifecycle maintain karo.** **4. Logout/revocation properly implement karo.** **5. Sensitive actions ke liye**

		7. Deep-link authentication: protected deep link ko unauthenticated state mein open karke behavior dekho.	6. Do not use biometrics as the sole authorization mechanism; use secure platform APIs.	**reauthentication/step-up authentication use karo.** **6. Biometric ko authorization ka sole replacement na banao; secure platform APIs use karo.**
M3:2024	**Insecure Communication**	**1. Cleartext traffic:** HTTP endpoints identify karo. **2. TLS validation:** certificate/hostname validation properly ho rahi hai ya nahi. **3. Custom trust logic:** insecure custom certificate validation/trust managers inspect karo. **4. Mixed-content communication:** secure app ka koi request insecure channel par fallback karta hai ya nahi. **5. Sensitive traffic inspection:** credentials, tokens, PII encrypted transport se ja rahe hain ya nahi. **Android:** Network Security Configuration review **iOS:** ATS configuration aur networking implementation review.	**3. Insecure Communication** 1. Use HTTPS/TLS. 2. Properly enforce certificate and hostname validation. 3. Disable cleartext traffic unless explicitly required. 4. Avoid insecure custom trust managers. 5. Keep sensitive API communication on authenticated encrypted channels. 6. Disable development exceptions in production builds.	**1. HTTPS/TLS use karo.** **2. Certificate and hostname validation properly enforce karo.** **3. Cleartext traffic disable karo unless explicitly required.** **4. Insecure custom trust managers avoid karo.** **5. Sensitive API communications ko authenticated encrypted channel par rakho.** **6. Development exceptions ko production build mein disable karo.**
M4:2024	**Insecure Data Storage**	**1. Local database:** SQLite/Core Data files mein PII, tokens, passwords etc. check karo. **2. Preferences:** Android SharedPreferences / iOS UserDefaults inspect karo. **3. Files/cache:** app sandbox mein sensitive files/cache identify karo. **4. Logs:** Android Logcat/iOS logs inspect karo. **5. Screenshots:** sensitive screens screenshot/app-switcher preview mein expose ho rahe hain ya nahi. **6. Backups:** backup/restore behavior se sensitive data recover hota hai ya nahi. **7. Clipboard:** sensitive values unnecessarily clipboard mein ja rahe hain ya nahi.	**4. Insecure Data Storage** 1. Minimize sensitive data storage. 2. Use Android Keystore-backed encryption and iOS Keychain. 3. Properly encrypt/protect sensitive local databases and files. 4. Do not store secrets or PII in logs. 5. Configure a secure backup policy. 6. Minimize sensitive-data retention. 7. Control clipboard usage and screenshots according to risk.	**1. Sensitive data ko minimum rakho.** **2. Android Keystore-backed encryption aur iOS Keychain use karo.** **3. Sensitive local databases/files appropriately encrypt/protect karo.** **4. Logs mein secrets/PII mat store karo.** **5. Secure backup policy configure karo.** **6. Sensitive data ki retention minimize karo.** **7. Clipboard aur screenshots ko risk ke hisaab se control karo.**
M5:2024	**Insufficient Cryptography**	**1. Weak algorithm:** deprecated/weak crypto implementation identify karo. **2. Hardcoded encryption keys:** binary/source/config mein keys search karo. **3. Weak random generation:** security-sensitive tokens/keys ke liye predictable random functions use ho rahe hain ya nahi. **4. Static IV/nonce:** encryption implementation mein reuse/misconfiguration check karo. **5. Key storage:** keys app files/preferences mein plain stored to nahi. **6. Custom crypto implementation:** home-grown encryption identify karo. **7. Key lifecycle:** generation, storage, rotation/revocation review.	**5. Insufficient Cryptography** 1. Use platform cryptographic APIs. 2. Use strong, approved algorithms. 3. Store keys in secure hardware-backed/platform storage where appropriate. 4. Do not hardcode cryptographic keys. 5. Use a secure random number generator. 6. Implement nonce/IV handling correctly. 7. Avoid home-grown cryptography.	**1. Platform cryptographic APIs use karo.** **2. Strong, approved algorithms use karo.** **3. Keys ko secure hardware-backed/platform storage mein rakho where appropriate.** **4. Cryptographic keys hardcode mat karo.** **5. Secure random generator use karo.** **6. Nonce/IV handling correctly implement karo.** **7. Home-grown cryptography avoid karo.**
M6:2024	**Insecure Code Quality**	**1. Input validation review.** **2. Unsafe API usage:** deprecated/insecure platform APIs identify karo. **3. Memory/resource handling:** risky native-code patterns where applicable **4. Error handling:** sensitive information exception/error mein leak to nahi **5. WebView code:** unsafe JavaScript/interface handling inspect karo. **6. Native bridge:** Android JNI / iOS native interfaces ke trust boundaries inspect karo. **7. Static-analysis findings:** high-risk code patterns identify karo.	**6. Insecure Code Quality** 1. Adopt secure coding standards. 2. Integrate SAST/static analysis. 3. Replace dangerous or deprecated APIs. 4. Use input validation and safe output handling. 5. Tightly restrict WebView and native bridges. 6. Include code review and security testing in CI/CD.	**1. Secure coding standards adopt karo.** **2. SAST/static analysis integrate karo.** **3. Dangerous/deprecated APIs replace karo.** **4. Input validation + safe output handling use karo.** **5. WebView/native bridges tightly restrict karo.** **6. Code review + security testing CI/CD mein include karo.**
M7:2024	**Insufficient Binary Protections**	**1. Debuggable build:** production app debug-enabled hai ya nahi. **2. Binary analysis:** package ko decompile karke sensitive logic/strings identify karo. **3. Reverse-engineering resistance:** critical client-side logic unnecessarily exposed hai ya nahi. **4. Code signing:** application properly signed hai ya nahi. **5. Runtime integrity:** modified/tampered application ke against security controls review karo. **6. Sensitive logic location:** security-critical decisions entirely client-side hain ya nahi.	**7. Insufficient Binary Protections** 1. Keep production builds debug-disabled. 2. Use strong application signing. 3. Keep sensitive business/security decisions on the server side. 4. Use obfuscation as defense-in-depth where justified. 5. Implement integrity/tamper protections according to risk. 6. Do not store secrets in the client binary.	**1. Production builds ko debug-disabled rakho.** **2. Strong application signing use karo.** **3. Sensitive business/security decisions server-side rakho.** **4. Obfuscation ko defense-in-depth ke roop mein use karo where justified.** **5. Integrity/tamper protections risk ke according implement karo.** **6. Secrets ko client binary mein mat rakho.**
M8:2024	**Security Misconfiguration**	**1. Android Manifest / iOS entitlements review.** **2. Exported components:** Android Activities, Services, Receivers, Providers unnecessarily exported hain ya nahi. **3. Excessive permissions:** app unnecessarily high-risk permissions request kar rahi hai ya nahi. **4. Debug configuration:** debug flags/test endpoints enabled hain ya nahi. **5. Backup configuration:** app data backup policy secure hai ya nahi. **6. WebView settings:** unnecessary JavaScript/file access enabled hai ya nahi. **7. iOS URL schemes/deep links/entitlements:** overly broad or unsafe configuration.	**8. Security Misconfiguration** 1. Use least-privilege permissions. 2. Export Android components only when required. 3. Remove debug/test configurations from production. 4. Securely configure the backup policy. 5. Harden WebViews. 6. Keep iOS entitlements to the minimum required scope. 7. Maintain and review a configuration baseline.	**1. Least-privilege permissions use karo.** **2. Android components ko required hone par hi exported rakho.** **3. Debug/test configuration production se remove karo.** **4. Backup policy securely configure karo.** **5. WebView harden karo.** **6. iOS entitlements ko minimum required scope tak rakho.** **7. Configuration baseline maintain + review karo.**
M9:2024	**Insecure Data Storage — Note:** Current OWASP Mobile Top 10 2024 mein M9 specifically **Insecure Data Storage** nahi hai; **M9 = Improper Session Handling**. Older Mobile Top 10 versions ke saath confusion hota hai.		**9. Insecure Data Storage** 1. Minimize sensitive local data. 2. Use secure platform storage such as Android Keystore and iOS Keychain. 3. Encrypt/protect sensitive databases and files. 4. Protect logs from sensitive-data exposure. 5. Secure backup and restore mechanisms. 6. Minimize data retention.	**Session security ko dedicated control ke roop mein test karo:** secure token storage, expiration, revocation, logout invalidation, reauthentication aur session binding. (OWASP Developer Guide)
M9:2024	**Insecure Session Handling**	**1. Logout test:** logout ke baad old token/session reuse. **2. Session timeout:** inactivity ke baad session remains valid ya expire hota hai. **3. Token rotation:** login/privilege change ke baad token rotate hota hai ya nahi. **4. Concurrent sessions:** old devices/sessions ko revoke karne ka mechanism hai ya nahi. **5. Device change:** sensitive session new/untrusted device context mein unnecessarily valid to nahi. **6. Biometric unlock:** local unlock ko server-side session authorization se incorrectly equate to nahi kiya gaya.		**1. Short-lived sessions use karo.** **2. Logout par tokens/session invalidate karo.** **3. Sensitive events par session/token rotate karo.** **4. Refresh tokens securely protect karo.** **5. Device/session management provide karo where needed.** **6. Sensitive actions par reauthentication/step-up authentication use karo.**
M10:2024	**Insufficient Security Controls**	**1. Runtime abuse controls review.** **2. Anti-tamper/integrity controls where justified.** **3. Root/jailbreak risk handling review.** **4. Debugger/runtime instrumentation exposure assessment in authorized test builds.** **5. Fraud/abuse controls:** sensitive functionality ko automated misuse se protect kiya gaya hai ya nahi. **6. Paid/protected functionality:** client-side checks alone par depend to nahi. **7. Defense-in-depth review:** authentication, authorization, integrity, monitoring sab independent layers mein hain ya nahi.	**10. Insufficient Cryptography** 1. Use platform cryptographic APIs. 2. Use strong, approved algorithms. 3. Store cryptographic keys securely. 4. Do not hardcode keys. 5. Use secure random number generation. 6. Implement correct IV/nonce handling. 7. Avoid custom cryptographic implementations.	**1. Security controls ko layered/defense-in-depth approach mein implement karo.** **2. Critical operations server-side enforce karo.** **3. Runtime integrity controls risk-based rakho.** **4. Abuse/fraud monitoring implement karo.** **5. Sensitive functionality ko client-only checks se protect mat karo.** **6. Security events monitor/log karo.** **7. Controls ko regular security testing se**

From <https://chatgpt.com/c/6a9eca64-2c24-83ee-ace2-797d21d9eaa6>